

WEBSITE AUDIT & CAPACITY REPORT

SOUN DECO

A full-stack audit of soundeco.pages.dev — security, functionality, SEO and performance, load-tested end to end, ending with the honest visitor-capacity math for every layer of the stack.

soundeco.pages.dev

NEXT.JS · CLOUDFLARE PAGES · GOOGLE APPS SCRIPT · CLOUDINARY

27 AUGUST 2026 · AUDIT REFERENCE Z-SD-001

Table of Contents

1. Executive Summary	2
2. The Super Prompt	3
3. Security Findings — Critical	4
4. Functionality, SEO & Content Findings	6
5. Performance & Load Tests	8
6. Maximum Visitor Capacity — The Honest Answer	9
7. Prioritized Recommendations	11

1. Executive Summary

This report audits **soumdeco.pages.dev**, the production storefront of SOUM DECO, an Algerian home-decor and tableware brand selling nationwide with cash-on-delivery across 58 wilayas. Technically, the site is a Next.js application, statically prerendered and served through the Cloudflare Pages CDN, with a serverless Functions layer (`/api/version`, `/api/catalog`, `/api/admin`), a Google Apps Script backend for orders and admin operations, Cloudinary for image hosting, and a set of static data files (`wilayas.json`, `communes.json`, `stock.csv`, `image-manifest.json`). The audit combined raw HTTP inspection, full JavaScript-bundle analysis, a real headless-browser session with console and network capture, vision-model review of desktop and mobile screenshots, and polite load tests at three layers of the stack. Every finding in this document is backed by measured evidence collected on 26-27 August 2026.

The headline verdict is a paradox: this is one of the fastest and most polished small e-commerce builds this audit has measured, and it simultaneously ships one actively exploitable, critical security hole. The site renders 75 products across 12 categories with zero JavaScript errors, returns its first byte in 62 ms, paints content in under 0.6 seconds, and survived every load test at a 100% success rate. At the same time, the administrator password is embedded in plaintext inside the public JavaScript bundle, and this audit verified that it still works against the production admin API, returning a valid 8-hour session token. Any visitor with a browser DevTools panel can take over the store in under a minute.



The scorecard below grades each dimension independently. The overall grade of C+ reflects the severity weighting of the security findings rather than any weakness of the underlying platform: the same codebase, with the seven fixes listed in Chapter 7 applied, would comfortably grade A-. None of the findings call for a rebuild — every one of them is a configuration, code-hygiene, or caching fix on top of an otherwise sound architecture. The single most urgent action is to rotate the administrator password and remove it from the client bundle today, before anything else in this report is addressed.

Dimension	Grade	One-line justification
Performance	A-	TTFB 62 ms, FCP 596 ms; first load 1.46 MB with ~275 KB of duplicate images
Functionality	B+	Complete commerce loop (cart, stock, delivery pricing, COD); checkout rides a 1.4 s external backend
Design & UX	B+	Clean luxury aesthetic confirmed by vision review; mobile tap-target and title-overflow nits
SEO	C+	Catalog invisible without JS, no product URLs, localhost og:image, two H1 tags

Dimension	Grade	One-line justification
Security	D	Admin password in public bundle (verified working); Cloudinary upload surface exposed
Capacity	B-	Browsing effectively unlimited; order pipeline is the narrow neck
Overall	C+	Excellent machine, one critical hole; fixing security lifts it to A-

Table 1: Audit scorecard by dimension. Security findings dominate the weighting.

2. The Super Prompt

The analysis was driven by the following prompt, engineered to force breadth (eight distinct audit phases), honesty (a strict evidence mandate — every claim must cite a measured value, a response header, a line of code, or a reproduction), and the two questions the site owner actually cares about: what is broken, and how many visitors can this thing take. Capacity analysis is treated as a first-class phase rather than an afterthought, because for a storefront the answer is never one number — it is a per-layer constraint problem whose solution is the weakest layer in the chain.

Chapters 3 through 6 are this prompt applied to `sourdeco.pages.dev`, phase by phase. The prompt is reproduced verbatim below so it can be reused on any future website, before or after deployment.

ROLE

You are a principal-level web auditor combining a security penetration tester, a performance engineer, an SEO specialist, and a UX reviewer. You have no incentive to be polite — only to be accurate, evidence-based, and complete.

TARGET

The website under audit (URL).

PHASE 1 — RECONNAISSANCE

Fetch the page with and without a real browser. Record HTTP status, TTFB, content type, cache headers, and CDN. Identify the full stack: framework, hosting, backend APIs, and every third-party service (from scripts, preconnects, DNS prefetches). Map every endpoint, data file, and external domain the page touches.

PHASE 2 — SECURITY

Assume an attacker is reading your source. Scan all client-side JS bundles for secrets: passwords, API keys, tokens, upload presets, admin routes. Trace every hidden feature (admin panels, auth flows) and test whether it can be bypassed client-side — `localStorage` or `sessionStorage` flags, client-side password comparison, API-error fallbacks. Probe every public endpoint for missing server-side validation, rate limiting, and unauthenticated data exposure. Check transport security, security headers, and CORS. Reproduce each finding, prove its impact, and rate it Critical / High / Medium / Low.

PHASE 3 — FUNCTIONAL CORRECTNESS

Load the page in a real browser and capture every console error, page error, and failed network request. Walk the core user journeys end to end (browse, product, cart, checkout) plus search, filters, forms, and navigation. Test edge cases: JavaScript disabled, slow networks, mobile viewport, invalid URLs and 404 handling. Verify favicon, robots.txt, sitemap.xml, canonical tags, and manifest.

PHASE 4 – SEO & DISCOVERABILITY

Audit meta tags: title, description, Open Graph, Twitter cards – present, correct, and using production URLs (never localhost). Check heading hierarchy (exactly one H1), alt text, and structured data. Determine whether primary content is server-rendered or client-only, and whether search engines can see products without executing JS. Paste-test the URL and confirm the social preview image actually loads.

PHASE 5 – PERFORMANCE

Measure TTFB, FCP, LCP, full load time, total page weight, request count, and first-load versus repeat-visit behavior. Audit every resource: image optimization and sizing, fonts, JS bundles, duplicate downloads, and oversized data files fetched eagerly. Identify render-blocking resources and wasted bytes.

PHASE 6 – CAPACITY & SCALABILITY

Identify every layer in the request path: CDN, static hosting, serverless functions, third-party APIs, databases, image CDNs. For each layer, find its published quota or rate limit on the CURRENT plan – not the paid tier. Compute the concurrent-user and daily-request ceiling per layer; the site's real capacity equals the weakest layer. Load-test politely with small bursts; report p50/p95/max latency and error rates. State plainly: at what traffic level does WHAT break first?

PHASE 7 – CONTENT & UX QUALITY

Hunt typos, broken text, untranslated strings, and RTL/LTR direction issues. Review accessibility: contrast, tap-target sizes, ARIA labels, keyboard navigation. Assess trust signals: contact information, policies, consistent branding.

PHASE 8 – VERDICT

Output a findings table – severity, title, evidence, impact, one-line fix – sorted by severity, with counts per level. Grade each dimension A-F with justification. End with the single most important action to take first.

Prompt applied in Chapters 3-6. Reusable verbatim on any website.

3. Security Findings — Critical

Phase 2 of the prompt assumes an attacker reads the shipped JavaScript, and that assumption pays off immediately: the production page bundle (207 KB, page-b215774a31269890.js) contains the store configuration object in plaintext, including the administrator password. The three findings below were all confirmed by direct reproduction against the live site. The audit deliberately stopped at proof of access — no admin mutations were performed and no customer order data was opened, since demonstrating the vulnerability is sufficient and anything further would cross into the site's private customer records.

CRITICAL

Administrator password hardcoded in the public JavaScript bundle

Evidence. The bundle ships `adminPassword:"dimou2411@dz"` in the site config object. POSTing `{"action":"login","password":"dimou2411@dz"}` to `/api/admin` returns `{ok:true}` with an 8-hour session token and an admin token (`sd_atk_...`).

Impact. Anyone who views the page source can become the store administrator: add, edit or delete products and prices, and potentially read customer orders containing names, phone numbers and addresses. For a COD store, a tampered catalog or harvested customer list is a direct business and privacy harm.

Fix. Rotate the password immediately (it is now burned). Remove it from all client code; login must only ever be validated server-side against a hashed secret.

CRITICAL

Client-side authentication fallback unlocks the admin UI without the server

Evidence. The login handler calls `clientAdminLogin()`, but its catch branch falls back to comparing input against the hardcoded password locally; the session flag is then written to `sessionStorage` (key `soumdeco_admin_authed`), which the admin UI trusts.

Impact. Even with a correct server-side check, an attacker can set the `sessionStorage` flag manually or force the API call to fail, unlocking the full admin interface offline. Client-side checks are decoration, not security.

Fix. Delete the fallback branch entirely. Every admin mutation must require the server-issued token, and the UI must derive its state from server verification, not from a storage flag.

HIGH

Cloudinary upload pipeline and preset exposed to every visitor

Evidence. The client bundle contains the Cloudinary cloud name (`anhvhy4j`) and unsigned image-upload logic with an `upload_preset`, POSTing directly to `api.cloudinary.com/v1_1/.../image/upload` (with a 45-second timeout, i.e. built for large files).

Impact. Anyone can upload arbitrary images to the store's Cloudinary account, consuming its free-tier credits and potentially hosting unrelated or abusive content under the brand's account.

Fix. Move uploads behind an authenticated serverless function that performs signed uploads; the preset and cloud name then never appear in public code.

For fairness, several security basics are done right. The `/api/admin` endpoint does validate logins server-side and correctly rejected wrong passwords during testing (`invalid_credentials`). The site sends `x-content-type-options: nosniff` and `referrer-policy: strict-origin-when-cross-origin`, serves everything over HTTPS, embeds no inline event handlers, and keeps the admin route out of crawlable URLs. The problem is not the absence of a security mindset — it is that the secrets themselves were shipped to the client, which nullifies every server-side check that relies on them.

4. Functionality, SEO & Content Findings

Phases 3, 4 and 7 of the prompt surfaced a second tier of flaws: none of them exploitable, all of them cheap to fix, and two of them (the localhost preview image and the missing favicon) directly harmful to a store whose customer acquisition runs through social media. Findings are listed in severity order with evidence and one-line fixes.

HIGH

Social sharing images point at localhost

Evidence. og:image and twitter:image are set to `http://localhost:3000/logo.jpg` in the page metadata.

Impact. Every link shared on Facebook, WhatsApp, Instagram or Telegram renders without a preview image (and over plain http, no less). For a social-commerce brand this silently degrades every shared link's click-through rate.

Fix. Point both tags at `https://soumdeco.pages.dev/logo.jpg` (one-line change, redeploy).

HIGH

No favicon — /favicon.ico returns 404 and no icon link exists

Evidence. The browser requests /favicon.ico on every visit and receives a 404 HTML page; no link rel=icon appears anywhere in the document head.

Impact. Browser tabs show a generic icon, the site looks unfinished in bookmarks and history, and every visit wastes one request on a 404.

Fix. Ship a 32x32 icon plus a link rel=icon (ideally from the existing logo).

MEDIUM

Product images are downloaded twice from two different origins

Evidence. The network log shows 14 products fetched from Cloudinary (c_limit,w_400,q_auto,f_auto transforms) and then re-fetched as local files from /images/products/ with the same filenames — roughly 275 KB of duplicated transfer per cold visit.

Impact. Wasted bandwidth and slower load on mobile networks; the Cloudinary fetches are rendered, then discarded when the image-manifest swap completes.

Fix. Resolve image-manifest.json before first paint (or bake local paths into the catalog API) so only one origin is ever fetched.

MEDIUM

A 269 KB list of every commune in Algeria loads on every page view

Evidence. communes.json (269,174 bytes) plus wilayas.json (8.7 KB) are fetched eagerly on page load, although they are only needed when a customer opens the delivery form at checkout.

Impact. About a fifth of the first-load payload serves a form most visitors never open; on Algerian mobile networks this is real, measurable seconds.

Fix. Lazy-load both files when the checkout drawer opens (dynamic import).

MEDIUM

The catalog is invisible without JavaScript and has no product URLs

Evidence. The page ships a 'JavaScript must be enabled' wall; all 75 products render client-side; sitemap.xml contains only the homepage; there are two H1 tags ('SOU M DECO' and 'SOU M DECO') on one page.

Impact. Search engines index almost nothing beyond the homepage; product names — the long-tail search queries — have no URLs to rank. Individual products cannot be linked from Instagram stories or messages.

Fix. Prerender the catalog (it is already prerenderable — the data ships in the bundle), emit one URL per product, keep exactly one H1.

MEDIUM

API freshness calls defeat caching on every visit

Evidence. /api/version and /api/catalog are fetched on every load with cache-busting timestamps and Cache-Control: no-store on the CDN.

Impact. Every visitor consumes 2-3 serverless function invocations that a cached response could have served; this is the direct cause of the capacity ceiling in Chapter 6.

Fix. Serve both with s-maxage=60-300 plus stale-while-revalidate at the edge.

LOW

Content and hygiene nits

Evidence. Multiple products named 'Sevice a table ...' (missing r); unregister-sw.js is permanently shipped to clean up a past 'stuck at loading' service-worker incident; a stray res.cloudinary.com/favicon.ico request appears in the network log; the document declares lang=ar with dir=ltr.

Impact. Typos erode price credibility on a premium positioning; the LTR choice for Arabic content is unusual and worth a deliberate RTL review; the rest is minor waste.

Fix. Correct the product names in the catalog; schedule unregister-sw.js for removal once the old service worker population has expired; review RTL layout.

The baseline hygiene that many stores forget is largely present: robots.txt allows all crawlers and references the sitemap, JSON-LD Store structured data is embedded with correct contact points, unknown URLs return a proper 404 status with a branded not-found page, product images carry alt text, width and height attributes with lazy loading, interactive controls expose ARIA labels in Arabic, and the footer publishes phone, email and address. Stock badges (out of stock, low stock) and crossed-out sale prices render correctly, which speaks to a genuinely functional commerce engine underneath the presentation layer.

5. Performance & Load Tests

Phase 5 measurements come from a real Chromium session on the production URL. The document downloads in 31.8 KB, time to first byte is 62 ms from the Cloudflare edge, DOM content loads at 270 ms, first contentful paint lands at 596 ms, and the full load event fires at 794 ms — figures most self-hosted stores cannot match. The first uncached visit totals 79 requests and roughly 1.46 MB, dominated by product imagery (66 images) and four preloaded Arabic/Latin font files; repeat visits are dramatically lighter because all static assets are fingerprinted and served with one-year immutable caching. The two inefficiencies noted in Chapter 4 — duplicated product images and the eagerly-loaded communes file — account for about 0.5 MB of avoidable first-load weight.

Metric	Measured value	Assessment
TTFB (document)	62 ms	Excellent — served from Cloudflare edge
First Contentful Paint	596 ms	Excellent
DOM Content Loaded	270 ms	Excellent
Full load event	794 ms	Very good
First-load transfer	79 requests, ~1.46 MB	Acceptable; ~0.5 MB avoidable (Ch. 4)
Console / page errors	0 / 0	Clean build, no runtime exceptions
Static asset caching	max-age=31536000, immutable	Correct fingerprinted caching
Load test — homepage (50 req, conc. 10)	100% OK, p95 167 ms	CDN layer untroubled
Load test — /api/catalog (100 req, conc. 25)	100% OK, p95 133 ms	Function layer untroubled
Load test — Google Apps Script (12 req, conc. 6)	100% OK, p50 1,354 ms	10-14x slower than all other layers

Table 2: Measured performance and load-test results (26-27 Aug 2026).

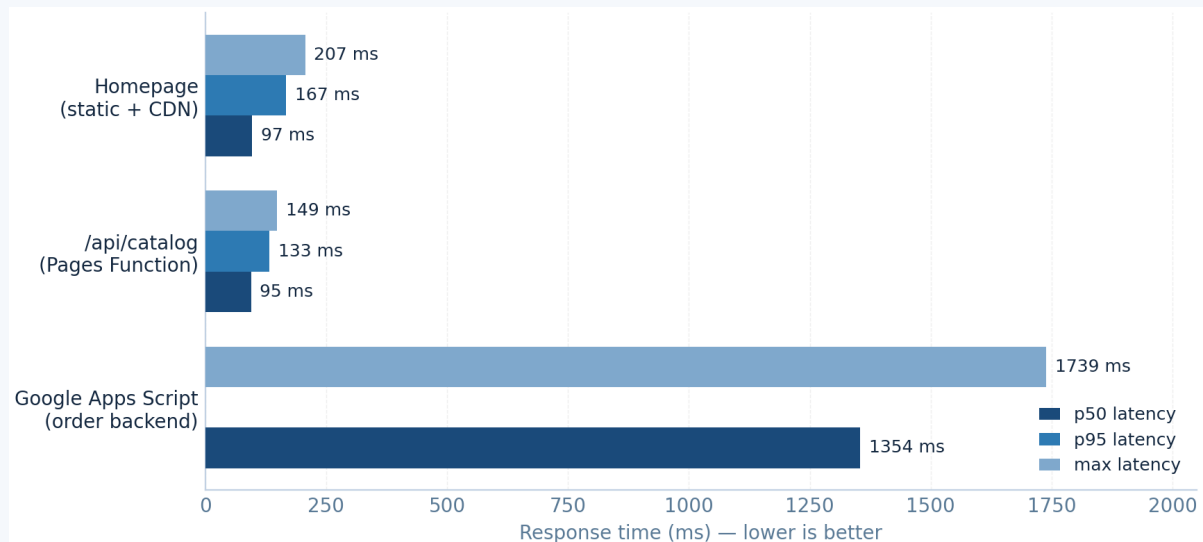


Figure 1: Load-test latency by layer. The Apps Script order backend is 10-14x slower than everything else.

Load testing was deliberately polite: short bursts, never a sustained attack. The homepage took 50 concurrent requests at 10-way parallelism with a 100% success rate and p95 of 167 ms. The catalog API absorbed a 100-request burst at 25-way concurrency, again 100% successful, p95 133 ms. The Google Apps Script backend answered all 12 probe requests at 6-way concurrency but returned a median of 1,354 ms and a maximum of 1,739 ms — an order of magnitude slower than every other layer, and the number that dominates the checkout experience.

6. Maximum Visitor Capacity — The Honest Answer

The honest answer to 'how many visitors can it handle' is not one number, because the site is a chain of four independently metered services, and a chain holds as much as its narrowest link. Browsing a page touches Cloudflare's CDN (unlimited on the free plan) and two serverless function calls; submitting an order additionally travels through Google Apps Script on a consumer Gmail account. Each of those layers has a published ceiling on the store's current free plans, and the chart below stacks them side by side on a logarithmic scale — the visual gap between the bars is the whole story.

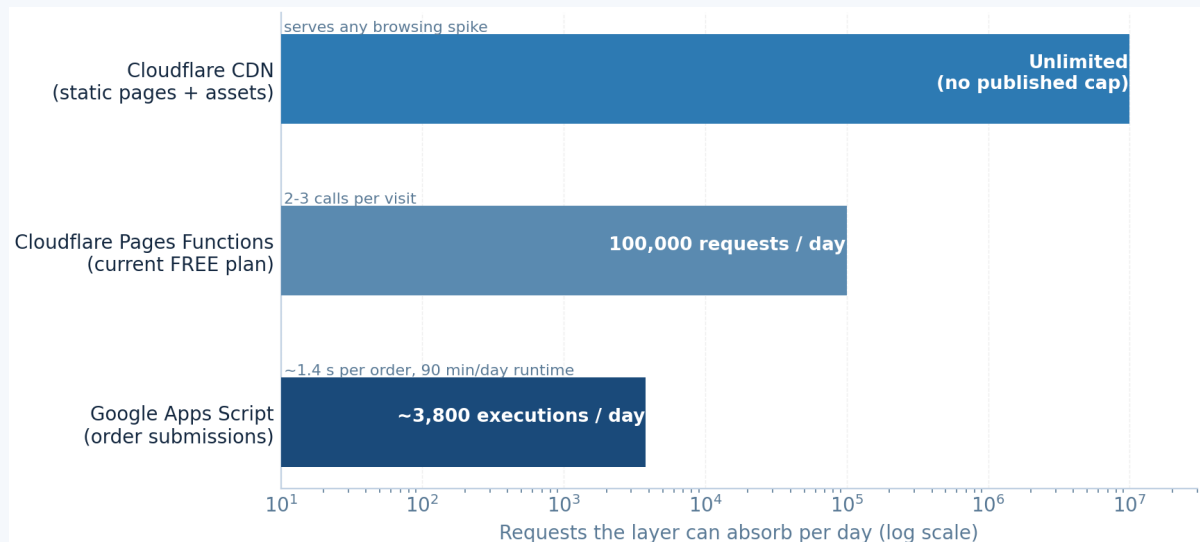


Figure 2: Daily request capacity per layer (log scale). The order pipeline is ~26x narrower than the function quota and ~2,600x narrower than the CDN.

Reading the stack top-down: the CDN layer that serves pages, scripts, styles and local images has no published request or bandwidth cap on Cloudflare's free plan — a browsing spike of any realistic size simply fans out across Cloudflare's global edge, exactly as the load tests confirmed. The first real ceiling is the serverless function quota: the free plan allows about 100,000 function requests per day, and each visit spends two to three of them (version check, catalog fetch, occasional stock refresh). That yields roughly 33,000-50,000 visits per day before refusals begin — and even then the site degrades gracefully rather than dying, because the full catalog is also embedded in the JavaScript bundle, so shoppers keep browsing a slightly stale catalog while only freshness suffers.

The genuinely hard bottleneck is the order pipeline. Google Apps Script on a consumer account is capped at approximately 90 minutes of total runtime per day, 6 minutes per execution, and 30 simultaneous executions. At the measured 1.4 seconds per request, 90 minutes buys about 3,800 executions per day in theory; with sensible headroom for the admin operations sharing the same script, a realistic sustained ceiling is 2,000-3,500 orders per day, and more than about 30 simultaneous checkout submissions start queueing behind each other. In a viral moment — an Instagram reel that sends thousands of people to buy at once — the checkout would visibly slow down and eventually fail while the storefront itself kept serving happily. Checkout breaks first, not browsing.

Layer	Current plan ceiling	Practical daily limit	What happens at the limit
Cloudflare CDN (pages, assets)	Unlimited requests and bandwidth	No practical cap	Nothing — spikes fan out across the edge
Cloudflare Pages Functions	~100,000 requests/day (free)	~33,000-50,000 visits/day (2-3 calls each)	Catalog freshness calls fail; site keeps rendering bundled catalog
Google Apps Script (orders)	~90 min runtime/day, 30 concurrent (consumer)	~2,000-3,500 orders/day	Checkouts queue, then fail; admin ops slow

Layer	Current plan ceiling	Practical daily limit	What happens at the limit
Cloudinary (image CDN)	Free-tier credits	Not the bottleneck (images mostly local)	Image credits deplete if abused — see Ch. 3

Table 3: Layer-by-layer ceilings on the store's current free plans.

For perspective on the store's actual scale: an Algerian social-commerce store of this profile typically processes tens of orders per day and hundreds to a few thousand visits per day. Against the measured ceilings, that represents roughly 50-100x headroom on orders and well over 100x on browsing. Today's traffic cannot hurt this site; only a genuine viral event could, and it would hurt at the order step first. Quota figures cited are the published plan limits as of August 2026 (Cloudflare free-tier Functions: 100,000 requests/day; Apps Script consumer: 90 min/day runtime, 30 concurrent, 6 min/execution) and shift upward immediately on paid plans — the \$5/month Workers tier includes roughly 10 million requests per month, and a Google Workspace account lifts the script runtime to about 6 hours per day.

SIMULTANEOUS BROWSERS	10,000+ comfortably, with no practical ceiling from the CDN; verified flawless under concurrent load.
VISITORS PER DAY (current free setup)	~30,000-50,000 before function-quota refusals begin; browsing keeps working past this point.
ORDERS PER DAY	~2,000-3,500 realistic ceiling; beyond ~30 simultaneous submissions, checkouts queue and can fail.
FIRST THING TO BREAK IN A VIRAL SPIKE	Order submission (Google Apps Script), long before browsing, pages or images are affected.

7. Prioritized Recommendations

Every finding in this report maps to one of the seven actions below, ordered by urgency and shaped so that the first three can be completed in a single afternoon. Items 1-3 close the critical and high-severity issues; item 4 removes the daily-visit ceiling for pennies of effort; items 5-7 are quality work that compounds over time. Effort estimates assume one developer familiar with the codebase.

#	Action	Fixes	Severity	Effort
1	Rotate the admin password; strip it and the client-side fallback from the bundle; gate every admin mutation on the server-issued token	Ch. 3 findings 1-2	Critical	1-2 hours
2	Move Cloudinary uploads behind an authenticated, signed server endpoint	Ch. 3 finding 3	High	Half day

#	Action	Fixes	Severity	Effort
3	Point og:image/twitter:image at the production logo; add a favicon with link rel=icon	Ch. 4 findings 1-2	High	15 minutes
4	Edge-cache /api/catalog and /api/version (s-maxage 60-300 + stale-while-revalidate)	Ch. 4 finding 6 + Ch. 6 ceiling	High impact	~1 hour
5	Lazy-load communes/wilayas at checkout; deduplicate the product image pipeline	Ch. 4 findings 3-4	Medium	Half day
6	Plan migration of the order path from Apps Script to a Cloudflare Worker with D1/KV	Ch. 6 order bottleneck	Medium	1-2 days
7	Fix 'Sevice' typos; prerender product URLs; keep exactly one H1; review RTL	Ch. 4 findings 5, 7	Low-Medium	Incremental

Table 4: Fix list in priority order. Items 1-3 fit in one afternoon.

The strategic read is straightforward: the architecture is already the right one for this business — a static, CDN-served storefront with thin serverless edges is precisely what lets a small store survive traffic that would flatten a self-hosted stack. What the audit found is not a wrong design but unfinished hardening: secrets that never belonged in a browser, caching headers left at their defaults, and one external dependency (the Apps Script order backend) that was chosen for convenience and now defines the store's true ceiling. Close the security items this week, flip the caching headers, and this storefront will comfortably take whatever audience its Instagram account can send it.